

Vision Transformer (ViT) 학습 가이드

Hugging Face 실습 기록

목차

1. [ViT CIFAR-10 훈련 소스코드 분석](#)
2. [허깅페이스 현업 사용 여부](#)
3. [SOTA\(State-of-the-Art\) 개념](#)
4. [ViT 추론\(Inference\) 코드 분석](#)
5. [허깅페이스 라이브러리의 확장성](#)
6. [커스텀 데이터셋 파인튜닝](#)

Q1. ViT CIFAR-10 훈련 소스코드 분석

전체 소스코드

```
# datasets는 huggingface에 속함
from datasets import load_dataset
import cv2
import numpy as np

# 전체 CIFAR10은 거의 70000개 (60000: train 10000:test)
# 부분적으로 다운로드함
train_ds, test_ds = load_dataset('cifar10', split=['train[:1000]', 'test[:200]'])

# 훈련 데이터를 훈련 + 검증으로 분할
splits = train_ds.train_test_split(test_size=0.1)
train_ds = splits['train']
val_ds = splits['test']

# 레이블 실제 이름 찾기
id2label = {id:label for id, label in enumerate(train_ds.features['label'].names)}
label2id = {label:id for id,label in id2label.items()}

from transformers import ViTImageProcessor
processor = ViTImageProcessor.from_pretrained("google/vit-base-patch16-224-in21k")

# 이미지 변환 설정
from torchvision.transforms import (CenterCrop, Compose, Normalize,
                                     RandomHorizontalFlip, RandomResizedCrop,
                                     Resize, ToTensor)

image_mean, image_std = processor.image_mean, processor.image_std
size = processor.size["height"]

normalize = Normalize(mean=image_mean, std=image_std)
_train_transforms = Compose([
    RandomResizedCrop(size),
    RandomHorizontalFlip(),
    ToTensor(),
    normalize,
])

_val_transforms = Compose([
    Resize(size),
    CenterCrop(size),
    ToTensor(),
    normalize,
])

def train_transforms(examples):
    examples['pixel_values'] = [_train_transforms(image.convert("RGB"))
                                for image in examples['img']]
    return examples

def val_transforms(examples):
    examples['pixel_values'] = [_val_transforms(image.convert("RGB"))
                                for image in examples['img']]
    return examples

# 데이터셋에 변환 설정
train_ds.set_transform(train_transforms)
val_ds.set_transform(val_transforms)
test_ds.set_transform(val_transforms)

from transformers import ViTForImageClassification

model = ViTForImageClassification.from_pretrained(
    'google/vit-base-patch16-224-in21k',
    id2label=id2label,
```

```

    label2id=label2id
)

from transformers import TrainingArguments, Trainer
from sklearn.metrics import accuracy_score

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return dict(accuracy=accuracy_score(predictions, labels))

args = TrainingArguments(
    f"test-cifar-10",
    save_strategy="epoch",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=1,
    num_train_epochs=3,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    logging_dir='logs',
    remove_unused_columns=False,
)

trainer = Trainer(
    model,
    args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    data_collator=collate_fn,
    compute_metrics=compute_metrics,
    tokenizer=processor,
)

trainer.train()

```

🔍 주요 함수 분석

함수 호출	소속 클래스/라이브러리	역할	리턴되는 객체
load_dataset('cifar10', ...)	datasets 라이브러리	Hugging Face 데이터셋 허브에서 CIFAR-10 데이터를 다운로드하고 기존 Dataset 을 지정된 비율로 무작위 분할하여 훈련/검증 데이터셋 생성	Dataset 객체의 리스트
train_ds.train_test_split(test_size=0.1)	datasets.Dataset 클래스	사전 학습된 ViT 모델에 맞는 이미지 전처리 설정 로드	DatasetDict 객체
ViTImageProcessor.from_pretrained(...)	transformers.ViTImageProcessor	사전 학습된 ViT 모델의 구조와 가중치 로드	ViTImageProcessor 인스턴스
ViTForImageClassification.from_pretrained(...)	transformers.ViTForImageClassification	사전 학습된 ViT 모델의 구조와 가중치 로드	ViTForImageClassification 인스턴스

💡 파이썬 딕셔너리 컴프리헨션

```
id2label = {id:label for id, label in enumerate(train_ds.features['label'].names)}
```

이 코드는 파이썬 딕셔너리 컴프리헨션(Dictionary Comprehension) 기법을 사용합니다.

- **구조:** {key_expression: value_expression for element in iterable}
- **역할:** 클래스 인덱스(ID)를 클래스 이름(Label)에 매핑하는 딕셔너리를 한 줄로 생성
- **예시:** {0: 'airplane', 1: 'automobile', ...}

🔧 데이터 변환 및 set_transform

set_transform의 역할

set_transform()은 데이터셋에 **즉석 변환(on-the-fly transformation)** 함수를 설정합니다. 데이터셋의 예제에 접근할 때 실시간으로 지정된 변환 함수가 자동으로 호출됩니다.

장점:

- 원본 데이터를 미리 변환하여 메모리에 저장할 필요가 없어 메모리 효율적
- 훈련 시 데이터 증강을 매 epoch마다 다르게 적용 가능

변환 파이프라인 구조

변환 파이프라인	구성 요소	역할	용도
_train_transforms	RandomResizedCrop(size)	이미지 크기를 무작위로 조정한 후 지정된 크기로 크롭	훈련 (데이터 증강)
	RandomHorizontalFlip()	이미지를 무작위로 수평 뒤집기	훈련 (데이터 증강)
	ToTensor()	PIL Image를 PyTorch Tensor로 변환	공통
	normalize	ViT 모델의 사전 학습 시 사용된 평균과 표준편차로 정규화	공통
_val_transforms	Resize(size)	이미지의 짧은 쪽을 지정된 크기로 먼저 크기 조정	검증/테스트
	CenterCrop(size)	중앙을 지정된 크기로 크롭 (이미지 일관성 유지)	검증/테스트
	ToTensor()	PIL Image를 PyTorch Tensor로 변환	공통
	normalize	픽셀 값 정규화	공통

⚙️ TrainingArguments 속성 설명

속성	설명	값
output_dir	모델 체크포인트와 로그가 저장될 경로	"test-cifar-10"
save_strategy	체크포인트를 저장할 기준	"epoch"
eval_strategy	평가(validation)를 수행할 기준	"epoch"
learning_rate	최적화에 사용할 학습률	2e-5 (0.00002)
per_device_train_batch_size	장치당 훈련 배치 크기	2
num_train_epochs	전체 데이터셋을 통과할 훈련 에포크 수	3
weight_decay	과적합 방지를 위한 가중치 감쇠(L2 정규화)	0.01
load_best_model_at_end	훈련 종료 시 가장 좋은 모델 체크포인트 로드 여부	True

📊 Confusion Matrix

****컨퓨전 매트릭스(Confusion Matrix, 혼동 행렬)****는 분류 모델의 성능을 시각화하고 평가하는 표입니다.

- **행(Row):** 실제 클래스 레이블 (True Label)
- **열(Column):** 모델이 예측한 클래스 레이블 (Predicted Label)
- **대각선:** 모델이 정확하게 예측한 경우의 수
- **비대각선:** 모델이 잘못 예측한 경우의 수

생성 과정

```
outputs = trainer.predict(test_ds)
y_true = outputs.label_ids # 실제 레이블
y_pred = outputs.predictions.argmax(1) # 예측된 확률에서 가장 높은 인덱스 선택

labels = train_ds.features['label'].names
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(xticks_rotation=45)
```

📁 저장된 모델 파일

파일명	내용	역할
pytorch_model.bin	모델의 가중치(weights)를 포함하는 PyTorch 상태 디렉터리	학습된 모델의 실제 지식(파라미터)
config.json	모델의 아키텍처 설정 파일	모델을 재구성하는 데 필요한 모든 메타데이터
preprocessor_config.json	이미지 전처리기의 설정 파일	모델이 기대하는 입력 형식을 정의하는 정보

👛 Q2. 허깅페이스 현업 사용 여부

네, Hugging Face의 **transformers** 및 **datasets**와 같은 라이브러리는 현재 인공지능 분야, 특히 자연어 처리(NLP) 및 비전(Vision) 분야의 현업에서 매우 널리 사용됩니다.

🏠 핵심 사용 이유

- ⚡ **효율성 및 생산성**: 사전 학습된 모델을 몇 줄의 코드로 쉽게 로드하고 파인튜닝할 수 있습니다.
- 🌐 **산업 표준 모델**: BERT, GPT, ViT 등 사실상의 산업 표준이 된 SOTA 모델 아키텍처와 가중치가 모두 공식적으로 지원됩니다.
- ✅ **일관된 API**: Trainer와 같은 통일된 고수준 API를 제공하여, 연구 결과물을 실제 서비스 가능한 코드로 전환하는 과정을 단순화합니다.
- 📄 **재현성**: 논문에서 제시된 결과를 다른 개발자나 팀원들이 쉽게 재현할 수 있도록 환경을 표준화합니다.

🆚 Hugging Face vs. From Scratch 구현 비교

구분	Hugging Face 라이브러리 사용	From Scratch 직접 구현
현업 사용 빈도	압도적으로 높음 (대부분)	매우 낮음 (특정 연구 개발 목적)
개발/구현 효율성	매우 높음 (몇 줄로 SOTA 모델 로드 및 훈련)	매우 낮음 (수백 줄의 복잡한 구현 필요)
모델 성능	최적화된 사전 학습 가중치 사용으로 빠르고 높은 성능 달성	초기 가중치부터 학습해야 하므로 대용량 데이터와 많은 시간 필요
디버깅 및 유지 보수	용이함 (커뮤니티 지원 및 공식 문서 활용)	어려움 (내부 로직 이해 및 직접 디버깅 필요)
커스터마이징	레이어, 설정 등 제한적인 커스터마이징	최대 수준의 자유도 (모든 모듈, 순서 변경 가능)

🏆 결론: 현업에서는 Hugging Face가 압도적입니다

현업의 목표는 대부분 "주어진 비즈니스 문제를 가장 효율적으로 해결하여 제품을 출시"하는 것입니다.

- 현업 (제품 개발 및 서비스)**: Hugging Face 라이브러리를 사용한 파인튜닝이 표준
- 연구 개발 (R&D)**: From Scratch 구현은 특정 목표를 위해서만 사용
 - 완전히 새로운 아키텍처를 제안하거나
 - 기존 아키텍처의 특정 모듈을 깊이 있게 분석하고 수정해야 할 때

🏆 Q3. SOTA(State-of-the-Art) 개념

SOTA는 특정 기술이나 소프트웨어 아키텍처를 가리키는 고유명사가 아닙니다. SOTA는 딥러닝이나 트랜스포머 같은 특정 분야에 한정되지 않고, **해당 시점에서 달성 가능한 가장 높은 수준의 성능 또는 결과**를 일반적/포괄적으로 지칭하는 용어입니다.

📚 SOTA의 정의

SOTA는 "State-of-the-Art"의 약자로, 우리말로로는 "**최첨단(기술)**" 또는 **"최고 성능"**으로 번역됩니다.

컴퓨터 비전(CV), 자연어 처리(NLP) 등 특정 분야의 공개적인 벤치마크 데이터셋 (예: ImageNet, GLUE, CIFAR-10)에서 **가장 좋은 성능 수치**를 기록한 모델, 방법론, 혹은 기술을 일컫습니다.

🔬 딥러닝/트랜스포머에서의 SOTA

딥러닝이나 트랜스포머 구조가 등장했을 때, 이들은 이전의 기술 대비 월등한 성능을 보여주었기 때문에 SOTA 기술로 불렸습니다.

기술/모델 아키텍처 예시:

- Transformer**가 등장했을 때, 순차적 데이터 처리(NLP) 분야의 SOTA를 차지했습니다.
- **ViT (Vision Transformer)****가 등장하여 CNN 기반 모델의 성능을 넘어서면서 이미지 분류 분야의 SOTA로 등극했습니다.

성능 지표 예시:

- 특정 모델이 GLUE 벤치마크에서 ****90.1%****의 정확도를 기록하여 이전 최고 기록을 갱신했다면, 그 모델이 해당 시점의 SOTA 모델이 됩니다.

💡 **핵심 이해:**

SOTA는 특정 기술이나 아키텍처 자체가 아니라, 그 기술이 제공하는 '최고의 결과'를 기준으로 부여되는 일종의 **칭호이자 상태**라고 이해하시면 됩니다.

🔍 Q4. ViT 추론(Inference) 코드 분석

📄 추론 소스코드

```
import torch
from PIL import Image
from transformers import ViTForImageClassification, ViTImageProcessor

MODEL_PATH = "./" # 모델이 저장된 폴더 경로

try:
    processor = ViTImageProcessor.from_pretrained(MODEL_PATH)
    model = ViTForImageClassification.from_pretrained(MODEL_PATH)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)
    model.eval() # 추론 모드로 설정
    print(f"✅ 모델 로드 완료. 현재 Device: {device}")
except Exception as e:
    print(f"❌ 모델 로드 중 오류 발생: {e}")
    exit()

IMAGE_FILE_PATH = "plane4.png"

try:
    image = Image.open(IMAGE_FILE_PATH).convert("RGB")
except FileNotFoundError:
    print(f"❌ 오류: 이미지 파일 '{IMAGE_FILE_PATH}'을(를) 찾을 수 없습니다.")
    exit()

# 이미지 전처리 및 텐서 생성
inputs = processor(images=image, return_tensors="pt").to(device)

with torch.no_grad():
    outputs = model(**inputs)

# 결과 해석
logits = outputs.logits
predicted_class_idx = logits.argmax(-1).item()

# 클래스 이름으로 변환
predicted_label = model.config.id2label[predicted_class_idx]

print("\n--- 추론 결과 ---")
print(f"예측된 클래스 인덱스: {predicted_class_idx}")
print(f"예측된 클래스 이름: **{predicted_label}**")
```

🔧 함수 호출 및 객체 분석

함수 호출	소속 클래스/라이브러리	역할	리턴되는 객체
<code>ViTImageProcessor.from_pretrained(MODEL_PATH)</code>	<code>transformers.ViTImageProcessor</code>	지정된 경로에서 이미지 전처리 설정을 로드하여 전처리를 초기화	<code>ViTImageProcessor</code> 인스턴스
<code>ViTForImageClassification.from_pretrained(MODEL_PATH)</code>	<code>transformers.ViTForImageClassification</code>	지정된 경로에서 모델 구조 설정 및 학습된 가중치를 로드하여 분류 모델을 초기화	<code>ViTForImageClassification</code> 인스턴스
<code>torch.device(...)</code>	<code>torch</code>	PyTorch가 연산을 수행할 장치	<code>torch.device</code> 객체

<code>model.to(device)</code>	<code>torch.nn.Module</code> 의 메서드	(CPU 또는 GPU)를 설정 모델의 모든 파라미터와 버퍼를 지정된 장치로 이동 모델을 추론	<code>model</code> 객체 (자기 자신)
<code>model.eval()</code>	<code>torch.nn.Module</code> 의 메서드	(evaluation) 모드로 설정 입력 이미지를 모델이 요구하는 형식으로 전처리하고 PyTorch 텐서 형식으로 반환	<code>model</code> 객체 (자기 자신)
<code>processor(images=image, return_tensors="pt")</code>	<code>ViTImageProcessor</code> 인스턴스	전처리된 텐서를 모델에 입력하여 순전파 연산을 수행 로짓 텐서에서 가장 큰 값을 갖는 인덱스를 찾음	<code>BatchFeature</code> 객체
<code>model(**inputs)</code>	<code>ViTForImageClassification</code> 인스턴스		<code>ImageClassifierOutput</code> 객체
<code>outputs.logits.argmax(-1).item()</code>	<code>torch.Tensor</code> 의 메서드		정수(int)

🚫 with torch.no_grad()의 사용 목적

`with torch.no_grad():`는 PyTorch에서 **추론(Inference)** 또는 **평가(Evaluation)** 시 필수로 사용되는 컨텍스트 매니저입니다.

역할:

이 블록 내에서 실행되는 모든 PyTorch 연산에 대해 **기울기(Gradient) 계산을 비활성화**합니다.

사용 목적:

- **메모리 절약:** 기울기 계산을 위한 중간 활성화 값을 저장할 필요가 없어 VRAM 사용량이 크게 줄어듭니다.
- **속도 향상:** 기울기 계산 자체가 생략되므로 연산 속도가 빨라집니다.
- **안전성:** 실수로 추론 과정에서 가중치가 업데이트되는 것을 방지합니다.

🔄 processor와 model을 별도로 가져오는 이유

processor의 역할

`processor`는 단순히 이미지를 텐서로 바꾸는 것을 넘어, **사전 학습 모델이 학습 당시 사용했던 정확한 전처리 과정**을 적용하는 역할을 합니다.

⚠️ 중요:

ViT 모델은 특정 평균(mean)과 표준편차(std)를 사용하여 정규화된 이미지를 입력으로 기대합니다. 이 정확한 값들은 모델의 가중치 파일에 포함되어 있지 않고, `preprocessor_config.json` 파일에 별도로 저장되어 있습니다.

만약 이 값이 잘못 적용된다면, 아무리 훌륭하게 학습된 모델이라도 전혀 다른 분포의 입력을 받게 되어 예측 성능이 급격히 떨어지게 됩니다.

📦 model.safetensors의 구조

포함 여부	내용	설명
포함됨	ViT의 파라미터	패치 임베딩, 포지션 임베딩, 트랜스포머 인코더의 모든 가중치, 분류 헤드의 가중치 등 수많은 텐서들
포함되지 않음	이미지 전처리 설정 평균, 표준편차, 크기 등	
포함되지 않음	모델의 아키텍처 설정	레이어 수, 헤드 수 등
포함되지 않음	레이블 매핑	id2label, label2id

이처럼 모델의 가중치 파일은 ****오직 숫자 배열(텐서)****만 담고 있으며, 텍스트 형식의 메타데이터(설정)는 config.json과 preprocessor_config.json 파일에 분리되어 저장됩니다.

Q5. 허깅페이스 라이브러리의 확장성

네, 사용자의 이해가 정확합니다. Hugging Face 라이브러리 구조의 핵심적인 장점은 **유연성과 확장성**에 있습니다. 새로운 기법이 등장했을 때, 전체 애플리케이션 코드를 수정할 필요 없이 핵심 로직만 추가하여 바로 통합하여 사용할 수 있습니다.

로직 일부 수정의 경우 (새로운 ViT 변형)

Vision Transformer (ViT)의 내부 로직 일부가 수정되는 경우(예: 새로운 어텐션 메커니즘, 더 나은 패치 임베딩)에는, Hugging Face의 해당 모듈을 ****상속(Inheritance)****받아 필요한 부분만 오버라이드하거나 새로운 클래스를 정의하여 기존 파이프라인에 주입할 수 있습니다.

장점: Trainer, ViTImageProcessor 등 기타 모든 유틸리티와 파이프라인은 그대로 유지됩니다.

아예 다른 인공지능 기법의 등장과 통합

아예 ViT와는 다른 **새로운 인공지능 기법**이 등장하더라도, Hugging Face의 transformers 라이브러리에 해당 기법을 새로운 클래스로 만들어 넣으면 애플리케이션 단에서는 **거의 수정 없이** 사용할 수 있습니다.

이것이 가능한 이유는 Hugging Face 라이브러리가 ****추상화(Abstraction)****를 통해 모든 모델을 일관된 인터페이스로 묶어놓았기 때문입니다.

요소	역할 및 일관성
모델 (AutoModel)	모델 아키텍처와 관계없이, forward() 메서드를 호출하면 로직을 반환합니다.
전처리기 (AutoProcessor)	모델 아키텍처와 관계없이, 입력 데이터를 모델이 요구하는 텐서로 변환합니다.
트레이너 (Trainer)	모델 클래스만 바꿔주면, 훈련 루프, 저장, 평가 로직은 변하지 않습니다.

새로운 모델 통합 시 필요한 구성요소

1. 새로운 모델 클래스 (예: NewModelForClassification)
2. 새로운 전처리기 클래스 (예: NewProcessor)
3. 새로운 환경 설정 파일 (config.json)

애플리케이션에서의 사용법

```
# 애플리케이션 코드 수정 없이 새로운 모델을 로드하여 사용 가능
from transformers import AutoModelForImageClassification, AutoProcessor

model = AutoModelForImageClassification.from_pretrained("huggingface/new-sota-model")
processor = AutoProcessor.from_pretrained("huggingface/new-sota-model")
# Trainer 사용법도 기존과 동일
```

통합 사례: ViT의 등장

Vision Transformer (ViT) 자체가 바로 이 사례에 해당합니다.

배경

딥러닝 초기에는 CNN(Convolutional Neural Network) 기반 모델(ResNet, VGG 등)이 이미지 분야의 SOTA였습니다.

ViT 통합

2020년에 ViT가 등장하면서 이미지 분야의 패러다임이 바뀌었지만, Hugging Face는 ViT를 **새로운 모델 클래스**로 추가했을 뿐, 기존 CNN 모델을 훈련/추론하는 데 사용되던 Trainer나 AutoProcessor의 사용 방식은 전혀 바뀌지 않았습니다.

결과

사용자는 CNN 모델 대신 ViT 모델을 선택하여 from_pretrained에 넣어주는 것만으로 새로운 아키텍처를 기존 애플리케이션에 통합할 수 있었습니다.

이러한 모듈화 및 표준화 덕분에 Hugging Face는 새로운 SOTA 모델이 등장할 때마다 라이브러리에 빠르게 통합하고, 사용자는 적은 노력으로 최신 기술을 활용할 수 있게 됩니다.

🎯 Q6. 커스텀 데이터셋 파인튜닝

제공해주신 코드는 이전 CIFAR-10 데이터셋을 사용한 코드와 **Vision Transformer (ViT) 모델을 파인튜닝하는 기본적인 파이프라인**은 동일하지만, **데이터 로딩 및 준비 과정**에서 커스텀 데이터셋을 사용하기 위한 중요한 변경 사항들이 적용되었습니다.

🔄 이전 코드와의 주요 변경점

1. 데이터셋 로드 방식 변경 (Custom Data 사용)

가장 큰 변화는 Hugging Face Hub에 있는 공개 데이터셋 대신 **로컬 커스텀 데이터셋**을 로드하는 부분입니다.

이전 (CIFAR-10)	현재 (커스텀 데이터)	설명
<code>load_dataset('cifar10', split=['train[:1000]', ...])</code>	<code>load_dataset('imagefolder', data_dir=CUSTOM_DATA_ROOT)</code>	imagefolder 로더를 사용하여 로컬 디렉터리의 폴더 구조를 Hugging Face Dataset 객체로 자동 변환
<code>train_ds.features['label'].names</code>	<code>raw_datasets['train'].features['label'].names</code>	imagefolder 로더는 폴더 이름 (예: 'can', 'pet', 'cup')을 기반으로 자동으로 클래스 레이블을 인식

핵심: 이 기법은 이미지 분류 문제를 위해 폴더 구조(각 폴더가 하나의 클래스를 대표)를 사용하는 일반적인 방식을 Hugging Face 파이프라인에 쉽게 통합할 수 있게 해줍니다.

2. 데이터셋 컬럼 이름 변경

CIFAR-10 데이터셋은 이미지 컬럼의 이름이 'img'였지만, imagefolder 로더를 통해 로컬 데이터를 로드하면 기본적으로 이미지 컬럼의 이름이 *****image*****로 설정됩니다.

이전 (CIFAR-10)	현재 (커스텀 데이터)	영향받은 함수
<code>examples['img']</code>	<code>examples['image']</code>	train_transforms 및 val_transforms 함수 내부에서 이미지 원본 컬럼 접근 방식이 변경됨

3. 클래스 개수(num_labels)의 명시적 설정

이전 CIFAR-10 코드는 10개의 클래스가 이미 모델에 내장되어 있거나 id2label을 통해 간접적으로 파악되었지만, 커스텀 데이터셋에서는 명시적으로 최종 분류 레이어의 크기를 지정해 줘야 합니다.

이전 (CIFAR-10)	현재 (커스텀 데이터)	설명
<code>num_labels</code> 변수 없음 (기본값 사용)	<code>num_labels = len(labels)</code> (예: 3)	ViTForImageClassification을 로드할 때 <code>num_labels=num_labels</code> 인수를 전달하여, 사전 학습된 모델의 최종 분류 레이어를 커스텀 데이터셋의 클래스 개수에 맞게 자동으로 재초기화
# 달라진 부분: num_labels 인수가 명시적으로 추가되어 분류 헤드를 커스텀 클래스 수에 맞춤 <code>model = ViTForImageClassification.from_pretrained(model_name_or_path, num_labels=num_labels, # <-- 이 부분이 중요 id2label=id2label, label2id=label2id)</code>		

4. 훈련 하이퍼파라미터 소폭 변경

커스텀 데이터셋의 크기에 맞춰 훈련 설정이 약간 변경되었습니다.

파라미터	이전 (CIFAR-10)	현재 (커스텀 데이터)	변경 이유
per_device_train_batch_size	2	10	CIFAR-10 데이터가 더 컸기 때문에 배치 크기를 낮게 설정했으나, 커스텀 데이터가 작아지면서 배치 크기를 늘려 GPU 활용도를 높임
num_train_epochs	3	5	데이터의 양이 적을수록 모델이 충분히 학습되도록 에포크 수를 늘리는 것이 일반적
output_dir	"test-cifar-10"	"custom-vit-finetune"	프로젝트 명칭에 맞게 변경

최종 결론

이 코드는 **Hugging Face 생태계의 유연성**을 보여줍니다. 즉, 복잡한 ViT의 아키텍처나 파인튜닝 로직(Trainer, compute_metrics, set_transform 내부의 전처리 파이프라인)은 그대로 유지하면서, **데이터 로드 방식과 클래스 개수만** 커스텀 환경에 맞춰 수정하면 재사용이 가능합니다.

Vision Transformer (ViT) 학습 가이드 - Hugging Face 실습 기록

이 문서는 Gemini와의 대화를 바탕으로 정리된 기술 문서입니다.